In [1]:
```python
# ========================
# IMPORT LIBRARIES
# ========================

import os
import json
import warnings
from collections import Counter

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import accuracy_score
from transformers import pipeline
import mlflow

warnings.filterwarnings("ignore")
```

In [2]:
```python
# ========================
# BASIC SETUP
# ========================

# Create folders for artifacts
os.makedirs("artifacts/task1_sentiment", exist_ok=True)
os.makedirs("artifacts/task2_ner", exist_ok=True)
os.makedirs("artifacts/task3_text_generation", exist_ok=True)
os.makedirs("screenshots", exist_ok=True)

# Set MLflow experiment
mlflow.set_experiment("Transformers_Downstream_Tasks")

print("Folders created successfully.")
print("MLflow experiment set to: Transformers_Downstream_Tasks")
```

```
2026/04/18 09:03:46 INFO mlflow.store.db.utils: Creating initial MLflow database tab
les...
2026/04/18 09:03:46 INFO mlflow.store.db.utils: Updating database tables
2026/04/18 09:03:46 INFO mlflow.tracking.fluent: Experiment with name 'Transformers_
Downstream_Tasks' does not exist. Creating a new experiment.
Folders created successfully.
MLflow experiment set to: Transformers_Downstream_Tasks
```

1□ Task 1 - Sentiment Classification

□ Goal

In this task, we use a pretrained transformer model for **sentiment analysis**.

Model

`distilbert-base-uncased-finetuned-sst-2-english`

What I will do

- create at least **20 custom sentences**
- provide **ground-truth labels**
- run inference using the Hugging Face **pipeline**
- calculate **accuracy**
- save predictions as a **CSV artifact**
- log everything in **MLflow**

We will perform **two separate MLflow runs** for this task.

In [3]:
```
# ========================
# TASK 1 DATA
# ========================

sentiment_data = [
    {"text": "I absolutely loved this movie. It was amazing from start to finish.",
    {"text": "The food was terrible and the service was even worse.", "true_label":
     {"text": "This laptop works perfectly and the battery life is excellent.", "tru
     {"text": "I regret buying this product. It broke on the first day.", "true_labe
     {"text": "The customer support team was very helpful and kind.", "true_label":
     {"text": "The room was dirty and smelled awful.", "true_label": "NEGATIVE"},
     {"text": "I am very happy with the quality of this phone.", "true_label": "POSI
     {"text": "This was a complete waste of money.", "true_label": "NEGATIVE"},
     {"text": "The teacher explained everything clearly and patiently.", "true_label
     {"text": "I hated the ending of the series. It was disappointing.", "true_label
     {"text": "The book was inspiring and beautifully written.", "true_label": "POSI
     {"text": "The app crashes every time I try to open it.", "true_label": "NEGATIV
     {"text": "Our vacation was relaxing, fun, and memorable.", "true_label": "POSIT
     {"text": "The package arrived late and the item was damaged.", "true_label": "N
     {"text": "This restaurant serves delicious meals every time.", "true_label": "P
     {"text": "The headphones sound awful and feel very cheap.", "true_label": "NEGA
     {"text": "I enjoyed the workshop and learned a lot.", "true_label": "POSITIVE"}
     {"text": "The website is confusing and hard to use.", "true_label": "NEGATIVE"}
     {"text": "The new update made the software faster and smoother.", "true_label":
     {"text": "I am disappointed by how poor the build quality is.", "true_label": "
]

sentiment_df = pd.DataFrame(sentiment_data)
sentiment_df.head()
print(f"Total sentences: {len(sentiment_df)}")
```

```
Total sentences: 20
```

In [4]:
```
# ========================
# TASK 1 MODEL
# ========================

sentiment_model_name = "distilbert-base-uncased-finetuned-sst-2-english"

sentiment_classifier = pipeline(
```

```
    task="sentiment-analysis",
    model=sentiment_model_name,
    device=-1 # use CPU
)

print("Sentiment pipeline loaded successfully.")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOK
EN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/104 [00:00<?, ?it/s]
Sentiment pipeline loaded successfully.
```

In [5]:

```
# ========================
# TASK 1 - RUN 1
# ========================

run1_predictions = sentiment_classifier(
    sentiment_df["text"].tolist(),
    truncation=True
)

task1_run1_df = sentiment_df.copy()
task1_run1_df["predicted_label"] = [pred["label"] for pred in run1_predictions]
task1_run1_df["confidence_score"] = [pred["score"] for pred in run1_predictions]

task1_run1_accuracy = accuracy_score(
    task1_run1_df["true_label"],
    task1_run1_df["predicted_label"]
)

task1_run1_csv = "artifacts/task1_sentiment/task1_run1_predictions.csv"
task1_run1_df.to_csv(task1_run1_csv, index=False)

with mlflow.start_run(run_name="Task1_Sentiment_Run1_DefaultPipeline"):
    mlflow.log_param("task", "sentiment_classification")
    mlflow.log_param("model_name", sentiment_model_name)
    mlflow.log_param("num_samples", len(task1_run1_df))
    mlflow.log_param("pipeline_type", "sentiment-analysis")
    mlflow.log_param("truncation", True)
    mlflow.log_param("device",  "cpu")

    mlflow.log_metric("accuracy", float(task1_run1_accuracy))

    mlflow.log_artifact(task1_run1_csv)

print("Task 1 - Run 1 completed.")
print("Accuracy:", task1_run1_accuracy)

task1_run1_df.head()
```

```
Task 1 - Run 1 completed.
Accuracy: 0.95
```

Out[5]:

	text	true_label	predicted_label	confidence_score
0	I absolutely loved this movie. It was amazing ...	POSITIVE	POSITIVE	0.999886
1	The food was terrible and the service was	NEGATIVE	NEGATIVE	0.999793
2	This laptop works perfectly and the battery li...	POSITIVE	POSITIVE	0.999835
3	I regret buying this product. It broke on	NEGATIVE	NEGATIVE	0.991246
4	The customer support team was very helpful and...	POSITIVE	POSITIVE	0.999758

In [6]:

```
# ========================
# TASK 1 - RUN 2
# ========================

# This run modifies two parameters compared to Run 1:
# truncation is kept True (ensures long inputs are safely handled)
# max_length is reduced to a smaller value to test if shorter input limits affect p
# This helps observe whether limiting input length changes model outputs or accurac

run2_predictions = sentiment_classifier(
    sentiment_df["text"].tolist(),
    truncation=True,
    max_length=32
)

task1_run2_df = sentiment_df.copy()
task1_run2_df["predicted_label"] = [pred["label"] for pred in run2_predictions]
task1_run2_df["confidence_score"] = [pred["score"] for pred in run2_predictions]

task1_run2_accuracy = accuracy_score(
    task1_run2_df["true_label"],
    task1_run2_df["predicted_label"]
)

task1_run2_csv = "artifacts/task1_sentiment/task1_run2_predictions.csv"
task1_run2_df.to_csv(task1_run2_csv, index=False)

with mlflow.start_run(run_name="Task1_Sentiment_Run2_MaxLength32"):
    mlflow.log_param("task", "sentiment_classification")
    mlflow.log_param("model_name", sentiment_model_name)
    mlflow.log_param("num_samples", len(task1_run2_df))
    mlflow.log_param("pipeline_type", "sentiment-analysis")
    mlflow.log_param("truncation", True)
    mlflow.log_param("max_length", 32)
    mlflow.log_param("device", "cpu")

    mlflow.log_metric("accuracy", float(task1_run2_accuracy))

    mlflow.log_artifact(task1_run2_csv)
```

```
print("Task 1 - Run 2 completed.")
print("Accuracy:", task1_run2_accuracy)

task1_run2_df.head()
```

```
Task 1 - Run 2 completed.
Accuracy: 0.95
```

Out[6]:

	text	true_label	predicted_label	confidence_score
0	I absolutely loved this movie. It was amazing ...	POSITIVE	POSITIVE	0.999886
1	The food was terrible and the service was even...	NEGATIVE	NEGATIVE	0.999793
2	This laptop works perfectly and the battery li...	POSITIVE	POSITIVE	0.999835
3	I regret buying this product. It broke on the ...	NEGATIVE	NEGATIVE	0.991246
4	The customer support team was very helpful and...	POSITIVE	POSITIVE	0.999758

In [7]:

```
# ========================
# TASK 1 SUMMARY TABLE
# ========================

task1_summary = pd.DataFrame({
    "run_name": [
        "Task1_Sentiment_Run1_DefaultPipeline",
        "Task1_Sentiment_Run2_MaxLength32"
    ],
    "accuracy": [task1_run1_accuracy, task1_run2_accuracy],
    "num_samples": [len(task1_run1_df), len(task1_run2_df)]
})

task1_summary
```

Out[7]:

	run_name	accuracy	num_samples
0	Task1_Sentiment_Run1_DefaultPipeline	0.95	20
1	Task1_Sentiment_Run2_MaxLength32	0.95	20

☐ Task 1 Observations

- The model achieved **95% accuracy** on 20 custom sentences in both runs.
- Run 2 used a reduced `max_length=32`, limiting how much input text the model processes.
- Despite this change, accuracy remained the same, indicating that shorter input length did not affect predictions for this dataset.

- MLflow successfully tracked both runs, including parameters, metrics, and prediction artifacts.

2□ Task 2 - Named Entity Recognition (NER)

□ Goal

In this task, we use a pretrained transformer model for **Named Entity Recognition**.

Model

`dslim/bert-base-NER`

What we will do

- create at least **10 custom paragraphs**
- run NER on each paragraph
- keep only:
  - **Person**
  - **Organization**
  - **Location**
- log:
  - model name
  - total entities extracted
  - distribution of entity types
- save the outputs as artifacts

We will perform **two separate MLflow runs** for this task.

In [8]:
```
# ========================
# TASK 2 DATA
# ========================

ner_paragraphs = [
    "Elon Musk announced that Tesla would build a new gigafactory near Berlin, Germ
    "Apple released its latest iPhone at a launch event in Cupertino. Tim Cook high
    "The United Nations Climate Summit in Paris brought together leaders from over
    "Researchers at MIT and Stanford published a joint paper on quantum computing.
    "Amazon expanded its warehouse operations in Manchester, England. The company p
    "President Biden met with Chancellor Scholz in Washington to discuss NATO expan
    "Google DeepMind achieved a breakthrough in protein structure prediction. Demis
    "Samsung unveiled new Galaxy devices at Mobile World Congress in Barcelona. The
    "The European Central Bank raised interest rates again. Christine Lagarde warne
    "Netflix announced a partnership with Studio Ghibli to stream their films globa
    "SpaceX launched another batch of Starlink satellites from Cape Canaveral, Flor
```

```
]

ner_df = pd.DataFrame({"paragraph": ner_paragraphs})
ner_df
print(f"Number of paragraphs: {len(ner_df)}")
```

```
Number of paragraphs: 11
```

In [9]:

```
# ========================
# TASK 2 MODEL
# ========================

ner_model_name = "dslim/bert-base-NER"

ner_pipeline_run1 = pipeline(
    task="ner",
    model=ner_model_name,
    aggregation_strategy="simple",
    device=-1  # CPU
)

ner_pipeline_run2 = pipeline(
    task="ner",
    model=ner_model_name,
    aggregation_strategy="first",
    device=-1  # CPU
)

print("NER pipelines loaded successfully.")
```

```
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
BertForTokenClassification LOAD REPORT from: dslim/bert-base-NER
Key                       | Status     | |
-------------------------+--------------------+--+-
bert.pooler.dense.weight | UNEXPECTED |  |
bert.pooler.dense.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok
if you expect identical arch.
Loading weights:   0%|          | 0/199 [00:00<?, ?it/s]
BertForTokenClassification LOAD REPORT from: dslim/bert-base-NER
Key                       | Status     | |
-------------------------+--------------------+--+-
bert.pooler.dense.weight | UNEXPECTED |  |
bert.pooler.dense.bias    | UNEXPECTED |  |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok
if you expect identical arch.
NER pipelines loaded successfully.
```

In [10]:

```
# ========================
# TASK 2 HELPER FUNCTION
# ========================
```

```python
def map_entity_label(entity_group):
    """
    Convert Hugging Face NER labels into the assignment categories:
    Person, Organization, Location
    """
    label = entity_group.upper()

    if label in ["PER", "PERSON"]:
        return "Person"
    elif label in ["ORG", "ORGANIZATION"]:
        return "Organization"
    elif label in ["LOC", "LOCATION"]:
        return "Location"
    else:
        return None


def run_ner_and_prepare_outputs(paragraphs, ner_pipeline_obj):
    records = []

    for idx, paragraph in enumerate(paragraphs, start=1):
        predictions = ner_pipeline_obj(paragraph)

        for pred in predictions:
            mapped_type = map_entity_label(pred.get("entity_group", ""))
            if mapped_type in ["Person", "Organization", "Location"]:
                records.append({
                    "paragraph_id": idx,
                    "paragraph": paragraph,
                    "entity_text": pred.get("word", ""),
                    "entity_type": mapped_type,
                    "score": float(pred.get("score", 0.0))
                })

    entity_df = pd.DataFrame(records)

    if len(entity_df) == 0:
        distribution = {"Person": 0, "Organization": 0, "Location": 0}
    else:
        counts = Counter(entity_df["entity_type"])
        distribution = {
            "Person": counts.get("Person", 0),
            "Organization": counts.get("Organization", 0),
            "Location": counts.get("Location", 0)
        }

    total_entities = sum(distribution.values())
    return entity_df, distribution, total_entities
```

In [11]:

```python
# ========================
# TASK 2 - RUN 1
# ========================

task2_run1_entities_df, task2_run1_distribution, task2_run1_total = run_ner_and_pre
    ner_df["paragraph"].tolist(),
    ner_pipeline_run1
```

```
)

task2_run1_csv = "artifacts/task2_ner/task2_run1_entities.csv"
task2_run1_json = "artifacts/task2_ner/task2_run1_distribution.json"

task2_run1_entities_df.to_csv(task2_run1_csv, index=False)

with open(task2_run1_json, "w", encoding="utf-8") as f:
    json.dump(task2_run1_distribution, f, indent=4)

with mlflow.start_run(run_name="Task2_NER_Run1_AggregationSimple"):
    mlflow.log_param("task", "named_entity_recognition")
    mlflow.log_param("model_name", ner_model_name)
    mlflow.log_param("num_paragraphs",  len(ner_df))
    mlflow.log_param("aggregation_strategy",  "simple")
    mlflow.log_param("entity_types_required", "Person, Organization, Location")
    mlflow.log_param("device", "cpu")

    mlflow.log_metric("total_entities_extracted",  int(task2_run1_total))
    mlflow.log_metric("person_count",  int(task2_run1_distribution["Person"]))
    mlflow.log_metric("organization_count", int(task2_run1_distribution["Organizati
    mlflow.log_metric("location_count",  int(task2_run1_distribution["Location"]))

    mlflow.log_artifact(task2_run1_csv)
    mlflow.log_artifact(task2_run1_json)

print("Task 2 - Run 1 completed.")
print("Total entities:", task2_run1_total)
print("Distribution:", task2_run1_distribution)

task2_run1_entities_df.head()
```

```
Task 2 - Run 1 completed.
Total entities: 40
Distribution: {'Person': 14, 'Organization': 15, 'Location': 11}
```

Out[11]:

	paragraph_id	paragraph	entity_text	entity_type	score
0	1	Elon Musk announced that Tesla would build a n...	Elon Musk	Organization	0.998025
1	1	Elon Musk announced that Tesla would build a n...	Tesla	Organization	0.571222
2	1	Elon Musk announced that Tesla would build a n...	Berlin	Location	0.999554
3	1	Elon Musk announced that Tesla would build a n...	Germany	Location	0.999700
4	2	Apple released its latest iPhone at a launch e...	Apple	Organization	0.998603

In [12]:

```
# ========================
# TASK 2 - RUN 2
# ========================
```

```
task2_run2_entities_df, task2_run2_distribution, task2_run2_total = run_ner_and_pre
    ner_df["paragraph"].tolist(),
    ner_pipeline_run2
)

task2_run2_csv = "artifacts/task2_ner/task2_run2_entities.csv"
task2_run2_json = "artifacts/task2_ner/task2_run2_distribution.json"

task2_run2_entities_df.to_csv(task2_run2_csv, index=False)

with open(task2_run2_json, "w", encoding="utf-8") as f:
    json.dump(task2_run2_distribution, f, indent=4)

with mlflow.start_run(run_name="Task2_NER_Run2_AggregationFirst"):
    mlflow.log_param("task", "named_entity_recognition")
    mlflow.log_param("model_name", ner_model_name)
    mlflow.log_param("num_paragraphs", len(ner_df))
    mlflow.log_param("aggregation_strategy", "first")
    mlflow.log_param("entity_types_required", "Person, Organization, Location")
    mlflow.log_param("device", "cpu")

    mlflow.log_metric("total_entities_extracted", int(task2_run2_total))
    mlflow.log_metric("person_count", int(task2_run2_distribution["Person"]))
    mlflow.log_metric("organization_count", int(task2_run2_distribution["Organizati
    mlflow.log_metric("location_count", int(task2_run2_distribution["Location"]))

    mlflow.log_artifact(task2_run2_csv)
    mlflow.log_artifact(task2_run2_json)

print("Task 2 - Run 2 completed.")
print("Total entities:", task2_run2_total)
print("Distribution:", task2_run2_distribution)

task2_run2_entities_df.head()
```

```
Task 2 - Run 2 completed.
Total entities: 35
Distribution: {'Person': 9, 'Organization': 15, 'Location': 11}
```

Out[12]:

	paragraph_id	paragraph	entity_text	entity_type	score
0	1	Elon Musk announced that Tesla would build a n...	Elon Musk	Organization	0.998970
1	1	Elon Musk announced that Tesla	Tesla	Organization	0.550648
2	1	Elon Musk announced that Tesla would build a n...	Berlin	Location	0.999554
3	1	Elon Musk announced that Tesla	Germany	Location	0.999700
4	2	Apple released its latest iPhone at a launch e...	Apple	Organization	0.998603

In [13]:
```
# ========================
# TASK 2 SUMMARY TABLE
# ========================

task2_summary = pd.DataFrame({
    "run_name":  ["Task2_NER_Run1_AggregationSimple",  "Task2_NER_Run2_AggregationFir
    "total_entities": [task2_run1_total, task2_run2_total],
    "person_count":  [task2_run1_distribution["Person"],  task2_run2_distribution["Pe
    "organization_count":  [task2_run1_distribution["Organization"],  task2_run2_dist
    "location_count":  [task2_run1_distribution["Location"],  task2_run2_distribution
})

task2_summary
```

Out[13]:

	run_name	total_entities	person_count	organization_count	locat
0	Task2_NER_Run1_AggregationSimple	40	14	15	
1	Task2_NER_Run2_AggregationFirst	35	9	15	

☐ Task 2 Observations

- The model successfully extracted entities from all paragraphs, limited to Person, Organization, and Location.
- Run 1 (`aggregation_strategy="simple"`) produced **more entities (40)** but included fragmented tokens (e.g., split names like "B", "##iden", "Schol", "##z").
- Run 2 (`aggregation_strategy="first"`) produced **fewer but cleaner entities (35)** by combining tokens into complete names (e.g., "Biden", "Scholz").
- This shows that aggregation strategy affects **entity quality and total count**, while the model remains the same.
- All runs were successfully tracked in MLflow with parameters, metrics, and artifacts.

3☐ Task 3 - Text Generation

☐ Goal

In this task, we use a pretrained generative transformer model for **text generation**.

Model

`gpt2`

What we will do

- prepare **5 prompts**

- generate text using different values for:
  - `temperature`
  - `max_length`
- log:
  - parameters used
  - generated outputs
  - observations as a text artifact

We will perform **two separate MLflow runs** for this task.

In [14]:
```
# ========================
# TASK 3 DATA
# ========================

generation_prompts = [
    "The future of artificial intelligence will",
    "In a world where technology controls everything,",
    "The biggest challenge facing humanity today is",
    "Deep in the ocean, scientists discovered",
    "Education in the next decade will be transformed by",
]

generation_df = pd.DataFrame({"prompt": generation_prompts})

generation_df
```

Out[14]:

	prompt
0	The future of artificial intelligence will
1	In a world where technology controls everything,
2	The biggest challenge facing humanity today is
3	Deep in the ocean, scientists discovered
4	Education in the next decade will be transform...

In [15]:
```
# ========================
# TASK 3 MODEL
# ========================

generation_model_name = "gpt2"

text_generator = pipeline(
    task="text-generation",
    model=generation_model_name,
    device=-1  # CPU
)

print("Text generation pipeline loaded successfully.")
```

```
Loading weights:   0%|          | 0/148 [00:00<?, ?it/s]
```

```
Text generation pipeline loaded successfully.
```

In [16]:

```python
# ========================
# TASK 3 HELPER FUNCTION
# ========================

def generate_text_outputs(prompts, generator, temperature, max_length, do_sample=Tr
    generated_records = []

    for i, prompt in enumerate(prompts, start=1):
        output = generator(
            prompt,
            max_length=max_length,
            temperature=temperature,
            do_sample=do_sample,
            num_return_sequences=1,
            pad_token_id=50256
        )[0]["generated_text"]

        generated_records.append({
            "prompt_id": i,
            "prompt": prompt,
            "generated_text": output
        })

    return pd.DataFrame(generated_records)
```

In [17]:

```python
# ========================
# TASK 3 - RUN 1
# ========================

task3_run1_temperature = 0.7
task3_run1_max_length = 50

task3_run1_df = generate_text_outputs(
    prompts=generation_df["prompt"].tolist(),
    generator=text_generator,
    temperature=task3_run1_temperature,
    max_length=task3_run1_max_length,
    do_sample=True
)

task3_run1_csv = "artifacts/task3_text_generation/task3_run1_generated_outputs.csv"
task3_run1_observation_file = "artifacts/task3_text_generation/task3_run1_observati

task3_run1_df.to_csv(task3_run1_csv, index=False)

task3_run1_observations_text = f"""
Task 3 - Run 1 Observations
Model: {generation_model_name}
Temperature: {task3_run1_temperature}
Max Length: {task3_run1_max_length}

Observation:
This run uses a moderate temperature and medium max length.
The generated text is usually fairly coherent while still showing some creativity.
```

```python
"""

with open(task3_run1_observation_file, "w", encoding="utf-8") as f:
    f.write(task3_run1_observations_text)

with mlflow.start_run(run_name="Task3_TextGeneration_Run1_Temp0.7_Max50"):
    mlflow.log_param("task", "text_generation")
    mlflow.log_param("model_name", generation_model_name)
    mlflow.log_param("num_prompts", len(generation_df))
    mlflow.log_param("temperature", task3_run1_temperature)
    mlflow.log_param("max_length", task3_run1_max_length)
    mlflow.log_param("do_sample", True)
    mlflow.log_param("device",  "cpu")

    mlflow.log_artifact(task3_run1_csv)
    mlflow.log_artifact(task3_run1_observation_file)

print("Task 3 - Run 1 completed.")
task3_run1_df
```

```
Passing `generation_config` together with generation-related arguments=({'temperatur
e', 'do_sample', 'pad_token_id', 'num_return_sequences', 'max_length'}) is deprecate
d and will be removed in future versions. Please pass either a `generation_config` o
bject OR all generation parameters explicitly, but not both.
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=50) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Task 3 - Run 1 completed.
```

Out[17]:

	prompt_id	prompt	generated_text
0	1	The future of artificial intelligence will	The future of artificial intelligence will be ...
1	2	In a world where technology controls everything,	In a world where technology controls everythin...
2	3	The biggest challenge facing humanity today is	The biggest challenge facing humanity today is...
3	4	Deep in the ocean, scientists discovered	Deep in the ocean, scientists discovered that ...
4	5	Education in the next decade will be transform...	Education in the next decade will be transform...

In [18]:

```
# ========================
# TASK 3 - RUN 2
# ========================

task3_run2_temperature = 1.0
task3_run2_max_length = 70

task3_run2_df = generate_text_outputs(
    prompts=generation_df["prompt"].tolist(),
    generator=text_generator,
    temperature=task3_run2_temperature,
    max_length=task3_run2_max_length,
    do_sample=True
)

task3_run2_csv = "artifacts/task3_text_generation/task3_run2_generated_outputs.csv"
task3_run2_observation_file  =  "artifacts/task3_text_generation/task3_run2_observati

task3_run2_df.to_csv(task3_run2_csv, index=False)

task3_run2_observations_text = f"""
Task 3 - Run 2 Observations
Model: {generation_model_name}
Temperature: {task3_run2_temperature}
Max Length: {task3_run2_max_length}

Observation:
This run uses a higher temperature and longer max length.
The outputs may become more creative and longer, but they can also become less cont
"""

with open(task3_run2_observation_file, "w", encoding="utf-8") as f:
    f.write(task3_run2_observations_text)

with mlflow.start_run(run_name="Task3_TextGeneration_Run2_Temp1.0_Max70"):
    mlflow.log_param("task", "text_generation")
    mlflow.log_param("model_name", generation_model_name)
    mlflow.log_param("num_prompts", len(generation_df))
    mlflow.log_param("temperature",  task3_run2_temperature)
```

```
    mlflow.log_param("max_length", task3_run2_max_length)
    mlflow.log_param("do_sample", True)
    mlflow.log_param("device",  "cpu")

    mlflow.log_artifact(task3_run2_csv)
    mlflow.log_artifact(task3_run2_observation_file)

print("Task 3 - Run 2 completed.")
task3_run2_df
```

```
Both `max_new_tokens` (=256) and `max_length`(=70) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=70) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=70) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=70) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Both `max_new_tokens` (=256) and `max_length`(=70) seem to have been set. `max_new_t
okens` will take precedence. Please refer to the documentation for more information.
(https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
Task 3 - Run 2 completed.
```

Out[18]:

	prompt_id	prompt	generated_text
0	1	The future of artificial intelligence will	The future of artificial intelligence will req...
1	2	In a world where technology controls everything,	In a world where technology controls everythin...
2	3	The biggest challenge facing humanity today is	The biggest challenge facing humanity today is...
3	4	Deep in the ocean, scientists discovered	Deep in the ocean, scientists discovered that ...
4	5	Education in the next decade will be transform...	Education in the next decade will be transform...

In [19]:

```
# ========================
# TASK 3 SUMMARY TABLE
# ========================

task3_summary = pd.DataFrame({
    "run_name": ["Task3_TextGeneration_Run1_Temp0.7_Max50", "Task3_TextGeneration_R
    "temperature": [task3_run1_temperature, task3_run2_temperature],
    "max_length": [task3_run1_max_length, task3_run2_max_length],
    "num_prompts": [len(generation_df), len(generation_df)]
})

task3_summary
```

Out[19]:

	run_name	temperature	max_length	num_prompts
0	Task3_TextGeneration_Run1_Temp0.7_Max50	0.7	50	5
1	Task3_TextGeneration_Run2_Temp1.0_Max70	1.0	70	5

☐ Task 3 Observations

- The GPT-2 model successfully generated text for all 5 prompts in both runs.
- Run 1 produced longer outputs but showed more repetition and less coherence in some cases.
- Run 2 generated more structured and meaningful text with improved readability.
- The difference is mainly due to changes in generation parameters (`temperature` and `max_length`), which control creativity and output length.
- This demonstrates how text generation quality can be influenced by parameter tuning while using the same pretrained model.
- All outputs and observations were logged as artifacts in MLflow.